

EXPT NO: 6	IMPLEMENTATION OF LSTM FOR SENTIMENT ANALYSIS
DATE:	

PRE-LAB QUESTIONS (PROVIDE BRIEF ANSWERS TO THE FOLLOWING QUESTIONS)

- What is the purpose of Long Short-Term Memory (LSTM) networks in NLP?**
LSTM networks are used to process and understand sequential text data by remembering important information from previous words. They can capture long-term dependencies, making them useful for tasks such as sentiment analysis, language translation, text generation, and speech recognition.
- Why is LSTM preferred over a Simple RNN for processing long text sequences?**
LSTM is preferred over a Simple RNN for long text sequences because it can retain important information for longer periods. Its memory cells and gates control what information should be stored, updated, or forgotten, reducing the vanishing gradient problem.
- What is sentiment analysis? Give two real-world applications.**
Sentiment analysis is the process of identifying the emotional tone or opinion expressed in text, such as positive, negative, or neutral. Applications include analyzing customer reviews to measure satisfaction and monitoring social media comments to understand public opinion.
- Explain the role of tokenization and padding in text preprocessing.**
Tokenization divides text into smaller units called tokens, such as words or subwords, so they can be processed numerically. Padding adds extra values to shorter sequences, making all input sequences equal in length for efficient batch processing.
- What is word embedding? Why is it useful for sentiment analysis?**
Word embedding represents words as numerical vectors that capture their meanings and relationships. It is useful for sentiment analysis because words with similar meanings have similar vector representations, helping the model understand positive, negative, and context-dependent words effectively.

AIM

To implement a Long Short-Term Memory (LSTM) network for sentiment analysis by performing text preprocessing, tokenization, sequence generation, and embedding using NLP libraries, and classify text into sentiment categories.

SOFTWARE REQUIREMENTS

- Python
- TensorFlow / Keras
- Google Colab
- GitHub
- NumPy
- Pandas
- Matplotlib
- NLP libraries – NLTK / spaCy
- Scikit-learn

OBJECTIVES

Students will be able to:

- Understand the architecture and working of LSTM networks.
- Perform text preprocessing techniques for sentiment analysis.
- Tokenize and convert textual data into numerical sequences.
- Generate word embeddings for textual data.
- Build and train an LSTM model for sentiment classification.
- Evaluate the performance of the trained LSTM model.

- Analyze the effect of preprocessing and embedding on sentiment classification.

PROCEDURE

PART A: DATASET PREPARATION AND TEXT PREPROCESSING

- Perform the following tasks:
- Load a sentiment analysis dataset such as the IMDB Movie Reviews dataset.
- Explore the dataset and examine sample positive and negative reviews.
- Check the number of samples and sentiment class distribution.
- Perform text preprocessing using NLP libraries.
- Convert text to lowercase.
- Remove unnecessary punctuation and special characters.
- Remove stop words, if required.
- Tokenize the reviews into words/tokens.
- Convert the tokens into numerical sequences using a tokenizer.
- Build a vocabulary from the training reviews.
- Apply sequence padding to obtain equal-length input sequences.
- Encode sentiment labels into numerical form.
- Split the prepared dataset into training and testing/validation datasets.

PART B: EMBEDDING GENERATION

Perform the following tasks:

- Generate numerical representations of the pre-processed text.
- Create an Embedding layer using Keras/TensorFlow.
- Define the vocabulary size and embedding dimension.
- Convert tokenized sequences into dense vector representations.
- Display the shape of the generated embedding representation.
- Analyze how words are represented in the embedding space.

PART C: IMPLEMENTING THE LSTM MODEL

Perform the following tasks:

- Create an input layer for the padded text sequences.
- Add an **Embedding layer** to generate dense word representations.
- Add an **LSTM layer** to learn sequential dependencies in the reviews.
- Add a **Dense output layer** with an appropriate activation function for sentiment classification.
- Compile the model using:
 - Appropriate optimizer
 - Binary cross-entropy loss
 - Accuracy as the evaluation metric
- Display the model architecture and summary.

PART D: MODEL TRAINING AND SENTIMENT PREDICTION

Perform the following tasks:

- Train the LSTM model using the prepared training dataset.
- Validate the model using the validation dataset.
- Record:
 - Training accuracy
 - Validation accuracy
 - Training loss
 - Validation loss
- Visualize:
 - Accuracy vs. Epoch

- Loss vs. Epoch
- Evaluate the trained model using the test dataset.
- Predict the sentiment of sample reviews.
- Compare the predicted sentiment with the actual sentiment.
- Calculate appropriate evaluation metrics such as:
 - Accuracy
 - Precision
 - Recall
 - F1-score
 - Confusion matrix

EXPECTED OUTCOME

Students will understand how LSTM networks learn sequential information from textual data and apply them to sentiment classification. Students will learn to perform text preprocessing, tokenization, sequence padding, and embedding generation using NLP libraries. They will build and train an LSTM model and evaluate its ability to classify reviews into positive and negative sentiments.

Screenshots:

The image displays two screenshots of a Jupyter Notebook interface. The left screenshot shows the notebook title "Lab-6 : LSTM-Based Sentiment Analysis" and the user's name "Pradeep Manikandan D" with roll number "24BAD088". The right screenshot shows the code for "PART A: DATASET PREPARATION AND TEXT PREPROCESSING", which includes loading the IMDB dataset, splitting it into training and testing sets, and printing the sample counts.

```
# PART A: Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
from sklearn.metrics import accuracy_score, precision_score, recall
```

```
# Load the built-in IMDB dataset
vocab_size = 10000
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=vocab_size)

print('Training samples:', len(x_train))
print('Testing samples:', len(x_test))
print('Positive samples:', np.sum(y_train == 1))
print('Negative samples:', np.sum(y_train == 0))
```

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb_data_vocabulary.pkl.gz

Training samples: 25000
Testing samples: 2500
Positive samples: 12500
Negative samples: 12500

File Edit Selection View Go Run ...

LSTM_Sentiment_Analysis_Pradeep_24BAD088.ipynb

Generate + Code + Markdown | Run All ...

Select Kernel

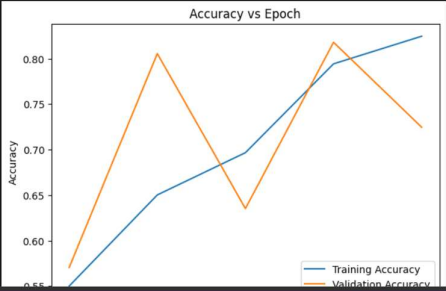
PART D: MODEL TRAINING AND SENTIMENT PREDICTION

```
# Train the model
history = model.fit(
    x_train, y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.2,
    verbose=1
)
```

Epoch 1/5
313/313 — 43s 132ms/step - accuracy: 0.5500 - loss: ...
Epoch 2/5
313/313 — 32s 101ms/step - accuracy: 0.6502 - loss: ...
Epoch 3/5
313/313 — 33s 104ms/step - accuracy: 0.6967 - loss: ...
Epoch 4/5
313/313 — 30s 94ms/step - accuracy: 0.7943 - loss: ...
Epoch 5/5
313/313 — 36s 115ms/step - accuracy: 0.8246 - loss: ...

```
plt.figure(figsize=(7, 5))
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('Accuracy vs Epoch')
plt.legend()
plt.show()
```

Accuracy vs Epoch



Epoch	Training Accuracy	Validation Accuracy
1	0.5500	~0.65
2	0.6502	~0.80
3	0.6967	~0.65
4	0.7943	~0.80
5	0.8246	~0.75

File Edit Selection View Go Run ...

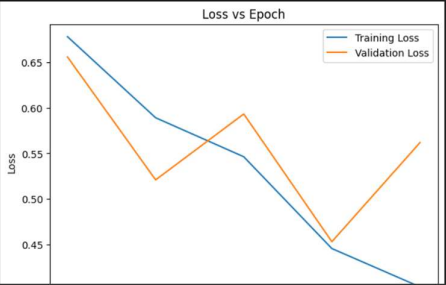
LSTM_Sentiment_Analysis_Pradeep_24BAD088.ipynb

Generate + Code + Markdown | Run All ...

Select Kernel

```
plt.figure(figsize=(7, 5))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Loss vs Epoch')
plt.legend()
plt.show()
```

Loss vs Epoch



Epoch	Training Loss	Validation Loss
1	~0.65	~0.55
2	~0.58	~0.52
3	~0.55	~0.58
4	~0.45	~0.52
5	~0.42	~0.55

```
# Evaluate the model on the test dataset
test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)
print('Test Loss:', test_loss)
print('Test Accuracy:', test_accuracy)
```

Test Loss: 0.548462510189478
Test Accuracy: 0.7312399744987488

```
# Calculate precision, recall, F1-score and confusion matrix
y_prob = model.predict(x_test, verbose=0)
y_pred = (y_prob >= 0.5).astype(int).flatten()

print('Accuracy:', accuracy_score(y_test, y_pred))
print('Precision:', precision_score(y_test, y_pred))
print('Recall:', recall_score(y_test, y_pred))
print('F1-score:', f1_score(y_test, y_pred))
print('\nClassification Report:\n')
print(classification_report(y_test, y_pred, target_names=['Negative', 'Positive']))
print('Confusion Matrix:\n', confusion_matrix(y_test, y_pred))
```

Accuracy: 0.73124
Precision: 0.8108398752554038

```
LSTM_Sentiment_Analysis_Pradeep_24BAD088.ipynb
Accuracy: 0.73124
Precision: 0.8108398752554038
Recall: 0.6032
F1-score: 0.6917748520574338

Classification Report:

              precision    recall  f1-score   support

   Negative       0.68       0.86       0.76       12500
   Positive       0.81       0.60       0.69       12500

 accuracy
macro avg       0.75       0.73       0.73       25000
weighted avg     0.75       0.73       0.73       25000

Confusion Matrix:
[[10741  1759]
 [ 4960  7540]]

# Predict sentiment for sample reviews
word_index = imdb.get_word_index()
word_index = {k: (v + 3) for k, v in word_index.items()}
word_index['<PAD>'] = 0
word_index['<START>'] = 1
word_index['<UNK>'] = 2
word_index['<UNUSED>'] = 3

def encode_review(text):
    words = text.lower().split()
    sequence = [1]
    for word in words:
        sequence.append(word_index.get(word, 2))
    return pad_sequences([sequence], maxlen=max_len, padding='post')

sample_reviews = [
    'This movie was excellent and I really enjoyed it',
    'The movie was boring and disappointing'
]

for review in sample_reviews:
    prediction = model.predict(encode_review(review), verbose=0)[0]
    sentiment = 'Positive' if prediction >= 0.5 else 'Negative'
    print('Review:', review)
    print('Predicted sentiment:', sentiment)
    print('Probability:', round(float(prediction), 4))
```

```
LSTM_Sentiment_Analysis_Pradeep_24BAD088.ipynb
for review in sample_reviews:
    prediction = model.predict(encode_review(review), verbose=0)[0][0]
    sentiment = 'Positive' if prediction >= 0.5 else 'Negative'
    print('Review:', review)
    print('Predicted sentiment:', sentiment)
    print('Probability:', round(float(prediction), 4))

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb_word_index.json
1641221/1641221 — 2s 1us/step
Review: This movie was excellent and I really enjoyed it
Predicted sentiment: Positive
Probability: 0.8505
Review: The movie was boring and disappointing
Predicted sentiment: Negative
Probability: 0.4132
```

GitHub Link:<https://github.com/Pradeep-Manikandan-1-D/24BAD088> **DEEP LEARNING LAB**
POST-LAB QUESTIONS (PROVIDE BRIEF ANSWERS TO THE FOLLOWING QUESTIONS)

- What is the role of the memory cell in an LSTM?**
The memory cell stores and carries important information across different time steps in a sequence. It helps the LSTM remember relevant information for a longer period and forget unnecessary information using its input, forget, and output gates.
- How does an LSTM overcome the vanishing gradient problem of a Simple RNN?**
LSTM uses a memory cell and gating mechanism to control the flow of information. The cell state provides a smoother path for gradients, allowing important information and gradients to be maintained over long sequences and reducing the vanishing gradient problem.
- What is the purpose of the Embedding layer in the sentiment analysis model?**
The Embedding layer converts words or integer tokens into dense numerical vectors. These vectors represent relationships and similarities between words, allowing the LSTM to understand the meaning and context of words and identify sentiment more effectively.

4. **How does padding affect the input to an LSTM network?**
Padding makes all text sequences the same length by adding zeros or special padding values to shorter sequences. This allows multiple sequences to be processed together in batches and provides a fixed-size input to the LSTM network.
5. **What happens when the LSTM model is trained for too many epochs?**
If an LSTM is trained for too many epochs, it may overfit the training data. The model starts learning noise and specific patterns from the training set, causing its performance on unseen or test data to decrease.
6. **Why are precision, recall, and F1-score useful in addition to accuracy?**
Precision, recall, and F1-score provide a more detailed evaluation of a classification model. They are particularly useful when the dataset has imbalanced classes, as accuracy alone may give a misleading impression of how well the model performs.
7. **Give one limitation of using LSTM for sentiment analysis.**
One limitation of LSTM is that it requires more computational resources and training time compared with simpler models. Since LSTM processes sequences step by step and contains several gates, training can become slower when dealing with large text datasets.

SUGGESTIONS FOR DATASETS

1. IMDB Movie Reviews – <https://ai.stanford.edu/~amaas/data/sentiment/>
2. SST-2 (Stanford Sentiment Treebank) – <https://huggingface.co/datasets/stanfordnlp/sst2>
3. Twitter Sentiment Analysis – <https://www.kaggle.com/c/twitter-sentiment-analysis>
4. Stanford Sentiment Treebank (SST) – <https://huggingface.co/datasets/stanfordnlp/sst>
5. Amazon Product Reviews – https://huggingface.co/datasets/amazon_polarity
6. Yelp Polarity Reviews – https://huggingface.co/datasets/fancyzhx/yelp_polarity

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	5	
In Lab Exercise	10	
Post Lab Exercise	5	
Viva	10	
Total	30	
Faculty Signature		